CUSTOMER SUPPORT RAG SYSTEM

Technical Architecture & Implementation Documentation

Document Summary

This document provides a comprehensive technical overview of the Customer Support Retrieval-Augmented Generation (RAG) system, covering its architecture, design decisions, workflow, testing strategy, challenges, and future roadmap.

26,000+	LangGraph	HITL
Knowledge Base Rows	Stateful Orchestration	Human-in-the-Loop

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is a hybrid AI paradigm that combines the reasoning capabilities of large language models (LLMs) with the precision of real-time document retrieval. Rather than relying solely on patterns learned during training, a RAG system actively fetches relevant documents from an external knowledge base at inference time and uses them as grounded context for generating responses.

This approach addresses one of the fundamental limitations of standalone LLMs: knowledge staleness and hallucination. By anchoring responses to retrieved, verifiable documents, RAG systems deliver factual accuracy that is impossible to achieve with static model weights alone.

The core RAG pipeline follows three stages:

- Encoding: The user query is converted into a dense vector embedding using a language model.
- Retrieval: The embedding is compared against a pre-indexed vector database to identify the most semantically similar document chunks.
- Generation: The retrieved chunks are injected into the LLM prompt as context, enabling it to generate a grounded, accurate response.

1.2 Use Case: Customer Support Assistant

This system implements a Customer Support Assistant designed to handle complex queries across refund processing, order status tracking, shipping policies, and returns management. The assistant references a knowledge base of over 26,000 rows, enabling it to provide precise, policy-compliant answers that reflect the latest operational guidelines.

Traditional rule-based support bots fail when queries are ambiguous or require multi-step reasoning. The RAG approach allows this system to understand context, retrieve the most relevant policy documents, and generate coherent, human-readable responses without requiring manual script updates.

Key business objectives served by this system:

- Reduce average handle time for support tickets by enabling instant, accurate self-service responses.
- Eliminate knowledge gaps caused by frequent policy updates, since retrieval is live and always reflects the latest data.
- Incorporate Human-in-the-Loop (HITL) escalation for ambiguous or high-stakes queries, ensuring quality assurance without sacrificing automation.
- Provide a scalable architecture that can be extended to new product lines or support categories without retraining the model.

2. System Architecture

2.1 Architecture Overview

The system is built on a Stateful Graph architecture, orchestrated by LangGraph. This design enables the query lifecycle to be modeled as a directed state machine, where each node represents a discrete processing step and transitions are governed by conditional logic. This architecture is particularly well-suited for support workflows that may require branching, looping, or human escalation.

Layer	Technology	Role
Frontend	React (Vite) + Aether Glass UI	User-facing chat interface with custom markdown rendering
Backend	FastAPI (Python)	REST API bridge between UI and LangGraph orchestrator
Orchestration	LangGraph (Stateful Graph)	Manages the full query lifecycle including HITL states
Vector Store	Embedding-backed Index	Stores and retrieves document chunks by semantic similarity
LLM	OpenAI / Compatible Model	Powers both the routing prompt and the generation prompt
Data Source	26,000+ row CSV / DB	Customer support knowledge base ingested in batches

2.2 Workflow Transition Diagram

The diagram below illustrates the complete query lifecycle through the stateful graph. Each node represents a distinct processing stage, and the directed edges represent conditional transitions based on routing logic.

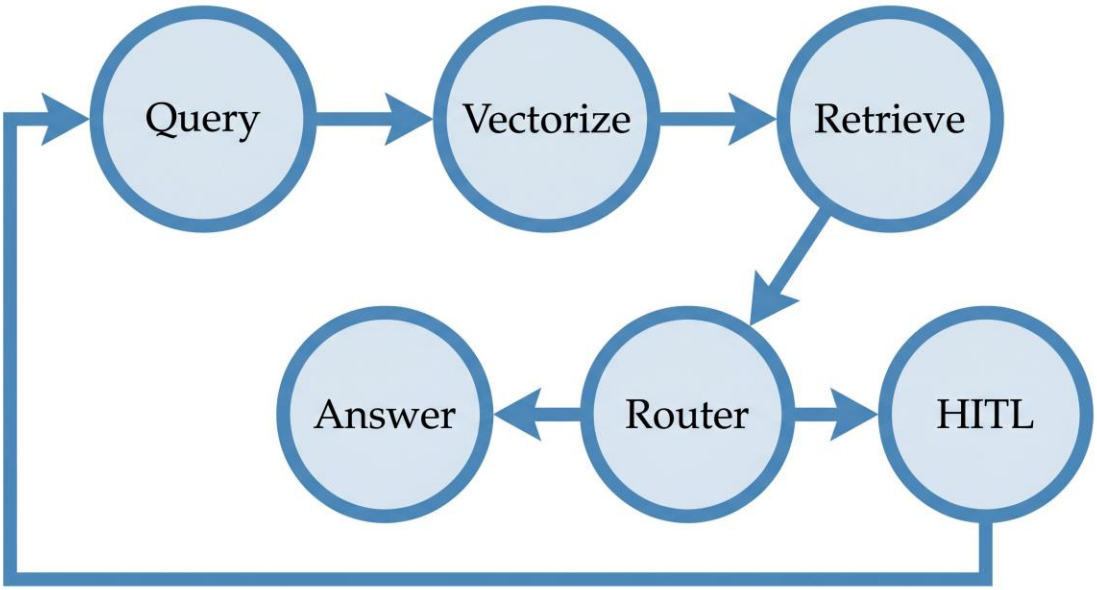

Figure 1: RAG System Workflow Transition Diagram

The workflow proceeds as follows:

- Query: The user submits a natural language support query via the frontend interface.
- Vectorize: The query is encoded into a dense embedding vector using a pre-trained language model.
- Retrieve: The vector is matched against the indexed knowledge base to fetch the top-N most relevant document chunks.
- Router: A specialized routing LLM call determines whether the retrieved context is sufficient to generate a reliable answer, or whether escalation is needed.
- Answer: If the context is sufficient, the generation LLM produces a formatted, grounded response and returns it to the user.
- HITL (Human-in-the-Loop): If the router determines the query is ambiguous, sensitive, or outside the knowledge base scope, the query is escalated to a human agent. Once resolved, the cycle restarts from the Query node.

3. Design Decisions

3.1 Chunk Size Strategy (1000 Tokens)

Document chunking is one of the most consequential design decisions in a RAG system. Chunk size directly affects both retrieval precision and generation quality.

A chunk size of 1,000 tokens was selected after empirical testing with the support knowledge base. This size is large enough to capture complete support instructions — including conditional clauses, policy exceptions, and multi-step resolution steps — while remaining small enough to fit 3 to 5 chunks within the LLM's effective context window without crowding out the generation prompt.

Smaller chunks (e.g., 200-300 tokens) led to fragmented instructions where critical conditions were split across chunks, causing incomplete or contradictory answers. Larger chunks (e.g., 2,000+ tokens) reduced retrieval precision by introducing topical dilution within each chunk.

3.2 Batched Ingestion Pipeline

The knowledge base contains over 26,000 rows of support data. Ingesting this dataset in a single synchronous operation would exceed typical API rate limits and connection timeouts for embedding models. The system addresses this with a batched ingestion pipeline in ingestion.py, which processes rows in configurable batch sizes and includes retry logic for transient failures.

This approach reduces peak memory consumption, allows for incremental updates to the knowledge base without full re-ingestion, and provides a natural checkpoint mechanism for monitoring ingestion progress.

3.3 Custom Markdown Rendering

The frontend required rich formatting for LLM responses, including bold emphasis, ordered lists, and code blocks. Rather than adopting a third-party markdown library — which would introduce dependency instability and versioning risks in the React (Vite) ecosystem — a lightweight custom markdown renderer was developed in-house.

This renderer handles the specific formatting patterns produced by the generation prompt, ensuring consistent visual output without external dependencies. It also enables fine-grained control over styling within the Aether Glass UI design system.

3.4 Dual-Prompt Strategy

The system employs two distinct LLM calls per query rather than a single monolithic prompt. This separation of concerns improves both accuracy and maintainability:

- Routing Prompt: A concise, specialized prompt that evaluates intent classification and context sufficiency. Its sole output is a routing decision: Answer or HITL. Keeping this prompt minimal reduces latency and improves decision consistency.
- Generation Prompt: A detailed instruction set that specifies response format, tone, citation requirements, and edge case handling. This prompt operates only when routing has confirmed that sufficient context exists.

This dual-prompt architecture prevents the routing logic from being influenced by generation instructions and vice versa, resulting in more reliable and auditable system behavior.

4. Challenges & Trade-offs

4.1 Knowledge Base Sampling vs. Recall

During early development, only 500 rows were ingested for speed, enabling rapid iteration on the retrieval and generation pipeline. However, this led to poor recall for specific query types — particularly refund queries involving precise monetary thresholds such as amounts around Rs. 7,000 — because the relevant policy rows were not present in the sampled index.

The decision was made to ingest 5,000+ rows despite the increased ingestion time. This trade-off prioritized answer accuracy over development velocity, which is the correct priority for a production support system where incorrect answers have direct customer impact.

Future mitigation strategies include stratified sampling during development (ensuring all policy categories are represented in the sample) and incremental ingestion with hot-reload capabilities for the vector index.

4.2 Port Configuration & Deployment Conflicts

Port 8000, the FastAPI default, is frequently occupied by other system services in standard development environments. This created intermittent startup failures during local development and CI/CD testing. The system was migrated to port 8080, which has lower contention in typical environments.

For production deployments, port configuration is externalized to environment variables to support containerized deployments where port mapping is handled at the infrastructure layer.

4.3 Retrieval Precision vs. Recall

The top-N retrieval parameter (number of chunks fetched per query) presents a fundamental precision-recall trade-off. A lower N increases precision but risks missing relevant context for complex multi-condition queries. A higher N improves recall but increases prompt size, latency, and the risk of context dilution where irrelevant chunks mislead the generation model.

The system is configured to retrieve 3 to 5 chunks per query, with the router prompt evaluating whether the retrieved set is sufficient before proceeding to generation. This dynamic sufficiency check acts as a quality gate, compensating for cases where retrieval precision falls short.

5. Testing Strategy

5.1 Unit Testing

Unit tests in ingestion.py verify that the embedding generation pipeline correctly transforms raw text rows into dense vector representations. Tests cover edge cases including empty strings, special characters, non-ASCII text (including Hindi and regional language content common in Indian customer support contexts), and maximum-length inputs.

5.2 Functional Testing

End-to-end functional tests were conducted via browser-based verification of the Aether Glass UI. Test scenarios included button interaction states, markdown rendering fidelity for complex LLM outputs, HITL escalation triggers and resolution flows, and session state management across multiple queries.

5.3 Accuracy Benchmarking

Manual accuracy benchmarking was performed using a curated set of queries with known expected answers. A key benchmark query involved a Rs. 7,000 refund scenario, which requires the system to correctly retrieve and apply tiered refund policy rules. This query was used as a regression test after each significant change to the ingestion pipeline, retrieval parameters, or generation prompt.

Planned future benchmarking includes automated evaluation using LLM-as-a-judge scoring, precision and recall metrics for the retrieval layer, and end-to-end accuracy scoring against a held-out test set of 200+ labeled queries.

6. Future Enhancements

6.1 Conversational Memory Integration

The current system treats each query as stateless. Adding LangGraph Checkpointer support will enable multi-turn conversational memory, allowing the assistant to reference previous exchanges within a session. This is critical for support scenarios where context builds across multiple messages, such as a customer following up on a previously submitted refund request.

6.2 Multi-Source RAG

The retrieval layer currently operates on a single structured knowledge base. Future iterations will enable simultaneous retrieval from heterogeneous source types including PDFs (product manuals, warranty documents), CSVs (order and transaction data), and live web documentation (shipping carrier APIs, policy microsites).

This multi-source capability will require source-aware chunk metadata tagging and a cross-source re-ranking step to prioritize the most authoritative sources for each query type.

6.3 Docker Containerization & Cloud Deployment

The system will be containerized using Docker, with separate service containers for the FastAPI backend, the LangGraph orchestrator, the vector store, and the React frontend. Docker Compose will manage local multi-service orchestration, while a Kubernetes-compatible configuration will support horizontal scaling in cloud environments.

Target deployment platforms include AWS ECS, Google Cloud Run, and Azure Container Apps. Environment-specific configuration will be managed through Kubernetes ConfigMaps and Secrets to maintain separation between application code and deployment parameters.

6.4 Monitoring & Observability

Production deployments will integrate structured logging with correlation IDs to enable end-to-end request tracing. Key metrics to be monitored include retrieval latency, generation latency, router decision distribution (Answer vs. HITL ratio), and per-query token consumption for cost management.

An LLM evaluation pipeline will run nightly against a golden dataset, automatically alerting the team if accuracy drops below defined thresholds following knowledge base updates or model version changes.